A HANDS-ON COURSE

Introduction to C

Compile, run, modify, recompile — and watch what changes.

```
$ gcc 01_basics.c -o basics && ./basics
```

7 C programs · 1 Makefile · 1 set of notes · 8 “try it” checkpoints

GETTING STARTED

The nine files

File	What it covers	What you do with it
01_basics.md	The notes: compiled language, types, arrays, functions, pointers, structures	Read it next to 01_basics.c
01_basics.c	The same topics as runnable code, labelled PRINT00 to PRINT23	Match every PRINT line to its code
Makefile	Recording the gcc commands; rebuilding only what changed	Run make, break it, extend it
02_array_pointers.c	arr, &arr[0] and &arr: one address, three types	Read the warnings, remove an &
03_char_strings.c	char, char* and char[]; strlen, strcpy, strtol	Uncomment a line, crash on purpose
04_argv.c	Command-line arguments	Run it with different arguments
05_write_files.c	fopen in “w” mode, fprintf, fclose	Run it, then look at file.txt
06_file_lines_to_string_array.c	fopen in “r” mode, fgets into a 2D char array	Create some_text.txt first
07_example_malloc_realloc.c	malloc, realloc and out-of-bounds writes	See a warning, a crash and a silent bug

GETTING STARTED

The working loop



Every program gets a “try it” slide

One line to change, what you should see once you recompile, and why it happens.

Keep two windows open

The editor on one side, a terminal in the folder of the files on the other. Recompile after every change.

GETTING STARTED

Commands you will type

Terminal, in the folder of the files

```
# compile and link into an executable named basics
$ gcc 01_basics.c -o basics
# same, with warnings switched on (do this)
$ gcc 01_basics.c -Wall -Wextra -o basics
# run it
$ ./basics
# exit status of the last program: 0 means success
$ echo $?
# send the output to a file instead of the screen
$ ./basics > out.txt
# run the Makefile / show a file / read a manual page
$ make
$ cat file.txt
$ man 3 strtol
```

Getting a compiler

Debian / Ubuntu: **sudo apt install build-essential**

macOS: **xcode-select --install** (gcc is an alias of clang; same commands)

Windows: use WSL, then the Ubuntu line

The -o option

Without it, gcc names the executable **a.out**. The Makefile in Part 2 names them **basics** and **array_pointers**.

Rule of the course

Read every warning. They are the course's teaching assistant.

GETTING STARTED

Four kinds of feedback

WARNING

```
warning: unused
variable ‘flower’
```

The compiler still produces the executable. Read it anyway: most bugs in this course announce themselves here.

ERROR

```
error: assignment of
read-only location
```

No executable is produced at all. The line and column tell you where to look. Fix, recompile.

CRASH

```
Segmentation fault
(core dumped)
```

It compiled and ran, then touched memory it must not. Exit status 139. “Aborted” (134) is its cousin.

SILENCE

```
BEFORE2: 10
```

Nothing complains, the output even looks right. Undefined behavior: the worst kind, and the one C is famous for.

Compiles without warnings and runs without crashing? That does not make it correct. Several programs in this course prove it.

PART 1 OF 6

01_basics.c

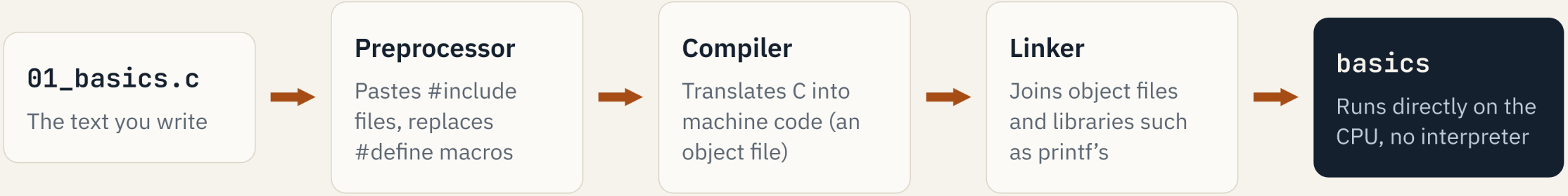
One program that touches every basic of the language, with its companion notes in 01_basics.md.

types · printing · casting · arrays · conditionals · functions · pointers · structures

```
$ gcc 01_basics.c -Wall -Wextra -o basics && ./basics
```

Needs foo_test.h in the same folder — its content is on the function-pointer slide.

A compiled language



Compiled: C

Fast, direct access to the hardware, a standalone executable. The price: recompile after every change, and an executable built on one platform does not run on another.

Interpreted: Python, JavaScript

An interpreter reads the source and executes it directly. More portable, no compile step, usually slower. The gcc command above is the step Python never asks you for.

Anatomy of a C program

hello.c

```c
#include <stdio.h>
int main() {
    printf("Hello, world!\n");
    return 0;
}
```

Terminal

```
$ gcc hello.c -o hello
$ ./hello
Hello, world!
$ echo $?
0
```

`#include <stdio.h>`

A preprocessor directive. It pastes in the standard input/output header, which declares printf.

`int main()`

The entry point. The operating system starts executing here; without main there is no program.

`printf(…); return 0;`

printf prints formatted text, \n ends the line. return 0 hands “success” back to the shell: echo $? shows it. Non-zero means an error.

Basic data types

Type	Holds	Usual size	In the course files
int	Whole numbers	4 bytes	int sum = 22, count = 3;
float	Real numbers, single precision	4 bytes	float pi = 3.1415927;
double	Real numbers, double precision	8 bytes	double mean;
char	One character	1 byte	char c = 'A';
bool	true or false — C99, needs #include <stdbool.h>	1 byte	bool statement1 = true;
typedef	Not a type: a new name for an existing one	—	typedef unsigned long int ulint;

Largest values, as noted at the top of 01_basics.c

int: 2,147,483,647 · unsigned int: 4,294,967,295 · long long int: 9,223,372,036,854,775,807, which is $2^{63} - 1$

“Usually” — ask your machine

```
printf("%zu %zu\n", sizeof(int), sizeof(double));
```

Sizes depend on the platform. sizeof returns them in bytes; %zu prints a size_t.

01_basics.c · LINES 1–7

Preprocessor directives

01_basics.c — top of the file

```
//preprocessor directives
#include <stdio.h> // printf
#include <stdbool.h>
#include "foo_test.h"
#define SIZE 4 // SIZE is a "macro"

typedef unsigned long int ulint;
```

See the preprocessor's work

```
$ gcc -E 01_basics.c | less
# thousands of lines: the headers pasted in,
# every SIZE replaced by 4, comments gone
```

`#include <stdio.h>`

Angle brackets: a system header, found in the compiler's folders. stdbool.h brings bool, true and false.

`#include "foo_test.h"`

Quotes: your own header, looked up in the current folder first. Written for this course; see the function-pointer slide.

`#define SIZE 4`

Text replacement before compilation. int my_arra2[SIZE] and the for loop both become 4 — change it once, it changes everywhere.

`typedef unsigned long int ulint;`

Not a directive but a declaration: ulint is now another name for unsigned long int.

01_basics.c · LINES 55–64

Printing to the screen

01_basics.c

```
printf("PRINT00\n");
/* Print format */
printf("PRINT01A: %10d\n", 12345);

float foobar = 0.9375;
printf("PRINT01B:%.2f\n", foobar);

/* puts(), fputs() */
puts("PRINT02");
fputs("PRINT03\n", stderr);
```

Terminal

```
$ ./basics
PRINT00
PRINT01A:      12345
PRINT01B:0.94
PRINT02
PRINT03
```

Format specifiers

%d integer · %ld long · %f float or double · %.2f two decimals · %10d right-aligned in 10 columns · %c character · %s string · %p address

puts adds the newline itself; fputs does not, and lets you choose the stream — here stderr, the error stream.

Integer division and type casting

01_basics.c

```
int sum = 22, count = 3;
float mean1;
mean1 = sum / count;
printf("PRINT04: Value of mean FLOAT : %f\n", mean1 );
int mean2 = sum / count;
printf("PRINT05: Value of mean INT : %d -------- %.12f\n", mean2, mean1 );
double mean;
mean = (double) sum / count;
printf("PRINT06: Value of mean FLOAT CAST: %f\n", mean );
```

```
PRINT04: Value of mean FLOAT : 7.000000
```

22 / 3 is computed between two ints: the result is 7, and only then stored in a float. C converts silently — “weakly typed”.

```
PRINT05: Value of mean INT : 7 -------- 7.000000000000
```

Same division stored in an int. The twelve decimals of %.12f prove that mean1 really holds 7.0, not 7.33.

```
PRINT06: Value of mean FLOAT CAST: 7.333333
```

(double) sum converts before the division happens: 22.0 / 3. The cast is explicit; the int-to-float conversions above were implicit.

TRY IT · CHECKPOINT 1 · 01_basics.c

Try it — printing and types

Change — line 76, in the PRINT05 printf: replace %d by %f. Recompile.

```
warning: format '%f' expects argument of type
'double', but argument 2 has type 'int'
```

Why — printf trusts the format string. A wrong specifier is undefined behavior: the number printed means nothing.

Change — delete line 44, the prototype **int andOne(int inputvalue);**

```
warning: implicit declaration of function 'andOne'
```

Why — main is compiled before andOne is defined, lower in the file. The prototype announces the function's signature in advance.

Change — nothing in the code. Run **./basics > out.txt**, then **cat out.txt**.

```
PRINT03 alone on the screen; every other line in
out.txt
```

Why — fputs(..., stderr) writes to the error stream. The > redirection only captures stdout.

Change — line 5: **#define SIZE 2**. Recompile, run.

```
PRINT07 lists 9 and 8 only — and no warning about
my_arra2[3] = 10
```

Why — the macro is replaced everywhere, loop included. Writing index 3 of a 2-element array is silent undefined behavior.

01_basics.c · LINES 90–103

Arrays

01_basics.c

```c
int my_arr1[] = {9, 8, 7, 6};
my_arr1[30] = 5; // Undefined Behavior

int my_arra2[SIZE] = {9, 8};
my_arra2[3] = 10;

int my_arr3[] = {9, 8, 7, 6};
printf("PRINT07: array loop:\n");
for (int i=0; i < SIZE; i++){
    printf("\t%d\n", my_arr3[i]);
}
```

Terminal

```
PRINT07: array loop:
        9
        8
        7
        6
```

Fixed size, one type

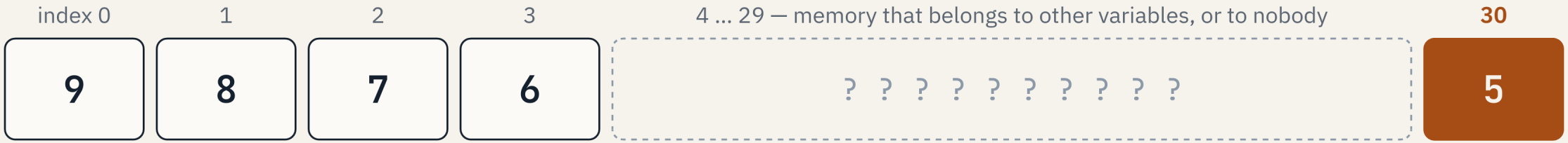
The memory is reserved when the program is compiled and never grows. Every element has the same type. {9, 8} in a 4-slot array leaves the last two at 0.

There is no .length: count with **sizeof(my_arr1) / sizeof(my_arr1[0])** — 16 bytes / 4 bytes = 4.

01_basics.c · LINE 91

No bounds checking

```
int my_arr1[] = {9, 8, 7, 6};
my_arr1[30] = 5; // Undefined Behavior (= nothing to interpret)
```



16 bytes were reserved. Index 30 is 120 bytes past the end — and the write goes through anyway.

Nothing checks the index

my_arr1[30] is simply the address of my_arr1 plus 30 × 4 bytes. C trusts you.

The outcome is undefined

Nothing visible here; a corrupted variable elsewhere; a crash. It can change with -O0 vs -O3, or from one machine to the next.

How to catch it

Compile with -fsanitize=address: the program then reports the offending line when it runs. Part 6 shows it.

Multidimensional arrays and iteration

01_basics.c — three equivalent 2D initializations, one 3D

```c
int c[2][3] = {{1, 3, 0}, {-1, 5, 9}};
//int c[][3] = {{1, 3, 0}, {-1, 5, 9}};
//int c[2][3] = {1, 3, 0, -1, 5, 9};

int test[2][3][4] = {
  {{3, 4, 2, 3}, {0, -3, 9, 11}, {23, 12, 23, 2}},
  {{13, 4, 56, 3}, {5, 9, 3, 5}, {3, 1, 4, 9}}
};
```

Arrays of arrays

c has 2 rows of 3 ints: c[1][2] is 9. In memory the 6 values sit one after the other, row after row. The first dimension may be left out when an initializer gives it.

01_basics.md — walking through an array

```c
int numbers[] = {1, 2, 3, 4, 5};
size_t length =
    sizeof(numbers) / sizeof(numbers[0]);

for (size_t i = 0; i < length; i++)
    printf("%d ", numbers[i]);

size_t j = 0;
while (j < length) {
    printf("%d ", numbers[j]);
    j++;
}
```

size_t, not int

sizeof returns a size_t, an unsigned type printed with %zu. Compare it with an int counter and -Wextra warns about “different signedness”.

Conditional statements

01_basics.c

```
bool statement1 = true;
bool statement2 = false;
if (statement1 && statement2){
    printf("PRINT08: Both are true\n");
}
if (statement1 || statement2){
    printf("PRINT09: One or more is true\n");
}
```

Terminal

```
PRINT09: One or more is true
```

&& is AND, || is OR, ! is NOT

PRINT08 never shows: true && false is false. Set statement2 to true, recompile, and both lines appear.

Zero is false, everything else is true

bool is only a convenience from stdbool.h. Comparisons (==, !=, <, >=) give 1 or 0, so if (count) and if (count != 0) mean the same thing.

The classic slip: = instead of ==

if (x = 5) assigns 5 and is always true. -Wall catches it: “suggest parentheses around assignment used as truth value”.

Functions and prototypes

01_basics.c — three places in the file

```c
// 1. the prototype, before main (line 44)
int andOne(int inputvalue);

// 2. the call, inside main (lines 134-136)
int test1 = 10;
int test2 = andOne(test1);
printf("PRINT10: %d → %d\n", test1, test2);

// 3. the definition, after main (206-209)
int andOne(int inputvalue){
    int outputvalue = inputvalue + 1;
    return outputvalue;
}
```

```
PRINT10: 10 → 11
```

Arguments are copied

test1 is copied into inputvalue. andOne works on its copy and returns a new value; test1 is still 10 afterwards. “Pass by value.”

Why the prototype

The compiler reads the file top to bottom. When it reaches the call, the definition is still 70 lines away; the prototype gives it the return type and parameters. Checkpoint 1 showed what happens without it.

Order of a C file

Directives → types and structures → prototypes → main → function definitions. 01_basics.c follows exactly this order; use it as a template.

Pointers: addresses and values

```c
printf("PRINT11: %p ....... %p\n", &test1, &test2);
int* ptr_test1 = &test1;
printf("PRINT12: %p\n", ptr_test1);
```

```
PRINT11: 0x7fffcbbe3028 ....... 0x7fffcbbe302c
PRINT12: 0x7fffcbbe3028
# yours differ, and change at every run
```



& gives the address

Every variable sits somewhere in memory. &test1 is that place, printed with %p. test2 sits 4 bytes further: one int.

int* declares a pointer

A pointer is a variable whose value is an address. It must be given a valid one before use; an uninitialized pointer is undefined behavior.

The pedantic %p

The standard defines %p for void*: printf("%p", (void*)&test1). gcc accepts int* without complaint; the cast is the portable form.

01_basics.c · LINES 148–152

Dereferencing

01_basics.c

```
// DEREFERENCING
printf("PRINT13: BEFORE = %d\n", test1);
(*ptr_test1)++;
// versus: *ptr_test1++; (precedence!)
printf("PRINT14: AFTER = %d\n", test1);
```

Terminal

```
PRINT13: BEFORE = 10
PRINT14: AFTER = 11
```

*** reads or writes through the pointer**

*ptr_test1 is the int that ptr_test1 points to, i.e. test1 itself. (*ptr_test1)++ increments test1 without naming it. The same * declares the pointer (int*) and dereferences it.

TRY IT NOW

Replace the highlighted line by ***ptr_test1++;** and recompile.

```
warning: value computed is not used
PRINT13: BEFORE = 10
PRINT14: AFTER = 10
```

++ binds tighter than *: the pointer itself is moved to the next int, and the old location is read and discarded. test1 never changes. Hence the parentheses.

Two meanings of *

In a declaration, int* p: “p is a pointer to int”. In an expression, *p: “the thing p points to”. In 03_char_strings.c the same applies to char*.

Passing by reference

01_basics.c

```c
void andOneMutability(int* ptr_inputvalue);

int test3 = 1000;
andOneMutability(&test3);
printf("PRINT15: %d\n", test3);

void andOneMutability(int* ptr_inputvalue){
    (*ptr_inputvalue)++;
}
```

```
PRINT15: 1001
```

BY VALUE

`andOne(test1)`

The function receives a copy of 10, returns 11. test1 is untouched.

BY REFERENCE

`andOneMutability(&test3)`

The function receives the address of test3 and modifies the original in place. It returns nothing: void.

Why this matters

A function can only return one value. Pointers let it change several variables of the caller — and avoid copying large data. Arrays, as Part 3 shows, are always passed this way.

Compared with Python

Python shares lists and copies nothing explicitly, but an int can never be changed by a callee. C makes the choice visible: pass the value, or pass its address with &.

Function pointers and header files

01_basics.c

```c
/* Define a function pointer */
int(*functionPointer)(int);
functionPointer = &andOne;

int result1 = (*functionPointer)(test3);
printf("PRINT16A: %d\n", result1);

int abc = 999999;
foo_function(&andOne, abc);
double test_math = sqrt(9);
```

```
PRINT16A: 1002
PRINT16B: 1000000
```

foo_test.h — the whole file

```c
#include <math.h>
int foo_function(int(*funcPtr)(int), int myvalue){
    int resulting_value = (*funcPtr)(myvalue);
    printf("PRINT16B: %d\n", resulting_value);
    return resulting_value;
}
```

return_type (*name)(parameter_types)

A variable that holds the address of a function. test3 is 1001 by now, so andOne gives 1002. Passing &andOne to foo_function hands it a behavior, not a value — the idea behind callbacks and custom sorting.

#include "foo_test.h" pastes the file in

So no prototype is needed, and math.h comes along: sqrt is available. If the linker ever complains about sqrt, add -lm to the gcc command.

Structures

01_basics.c — declared before main, after the directives

```c
typedef struct enemy Enemy; // alias
struct enemy{
    char name[100];
    int life;
    int strength;
};
typedef struct level Level;
struct level{
    int length;
    char name[100];
} Level_default = {23, "missing_name"};
typedef struct{
    int lifeup;
} Item;
```

A structure is a type, not a value

It groups members of different types under one name — a char array, two ints. Declaring struct enemy reserves nothing; declaring a variable of that type does.

Three spellings of the same idea

struct enemy boss; — the full name. typedef struct enemy Enemy; — an alias, declared before or after the struct. typedef struct {...} Item; — anonymous struct, alias only.

Level_default is a variable

Created right after the type, initialized member by member in order: length 23, name “missing_name”. Copying it is how the course gives a struct default values.

Structs nest, and replace dictionaries

A member can itself be a struct (an Address inside a Person). C has no hash table or dictionary: it is built out of structs and arrays.

Using structures

01_basics.c

```
Enemy boss1; // Instantiation
printf("PRINT17: %d\n", boss1.strength); //!!
boss1.strength = 9000;
printf("PRINT18: %d\n", boss1.strength);
Level level1 = Level_default;
printf("PRINT19: LEVEL LENGTH = %d\n",
    level1.length);
printf("PRINT20: LEVEL NAME = %s\n",
    level1.name);
Enemy* struct_pointer = &boss1;
printf("PRINT21: %d\n", boss1.strength);
printf("PRINT22: %d\n", (&boss1)->strength);
printf("PRINT23: %d\n",
    struct_pointer->strength);
```

```
PRINT17: 0
PRINT18: 9000
PRINT19: LEVEL LENGTH = 23
PRINT20: LEVEL NAME = missing_name
PRINT21: 9000
PRINT22: 9000
PRINT23: 9000
```

PRINT17 is a lie

boss1 was never initialized. The 0 is luck from -O3; with -O0 the same line prints 387300080, then something else on the next run. gcc warns: ‘boss1.strength’ is used uninitialized.

. for a struct, -> for a pointer to one

struct_pointer->strength is shorthand for (*struct_pointer).strength. Three spellings, one value: PRINT21, 22 and 23 all read the same memory.

TRY IT · CHECKPOINT 2 · 01_basics.c

Try it — pointers and structures

Change — the compile command, not the code: **gcc 01_basics.c -O0 -o basics**. Run it twice.

```
PRINT17: 387300080 — then PRINT17: -276264208
```

Why — boss1 is never initialized; you read whatever the memory held. Fix: **Enemy boss1 = {"Dragon", 100, 9000};** and PRINT17 becomes 9000.

Change — line 31: **Level_default = {42, "cave"}**

```
PRINT19: LEVEL LENGTH = 42 / PRINT20: LEVEL NAME = cave
```

Why — level1 = Level_default copies every member at once. A struct is assigned by value; an array, as Part 3 shows, is not.

Change — insert **struct_pointer->strength = 1;** just before PRINT21.

```
PRINT21: 1 / PRINT22: 1 / PRINT23: 1
```

Why — the pointer refers to boss1 itself. Writing through -> changes the original, exactly like (*ptr_test1)++ did.

Change — add **Enemy boss2 = boss1; boss2.strength = 5;** then print both strengths.

```
boss1: 9000, boss2: 5
```

Why — boss2 is a copy; the two structs live at different addresses. Compare with the previous card: a pointer shares, an assignment copies.

PART 2 OF 6

Makefile

One file that remembers your gcc commands and rebuilds only what changed.

variables | rules | targets and prerequisites | recipes | timestamps

```
$ make
```

Run it in the folder that holds the Makefile, 01_basics.c, 02_array_pointers.c and foo_test.h.

Makefile · THE WHOLE FILE

Anatomy of the Makefile

Makefile — recipe lines start with a real TAB character

```
CC = gcc
CFLAGS = -std=c11 -Wall -Wextra -O3 -march=native
SRC1 = ./01_basics.c
SRC2 = ./02_array_pointers.c
EXE1 = basics
EXE2 = array_pointers

# test
all: $(EXE1) $(EXE2)

$(EXE1): $(SRC1)
	$(CC) $(SRC1) $(CFLAGS) -o $(EXE1)

$(EXE2): $(SRC2)
	$(CC) $(SRC2) $(CFLAGS) -o $(EXE2)
```

Variables, expanded with $( )

$(CC) $(SRC1) $(CFLAGS) -o $(EXE1) becomes exactly the gcc command you typed by hand. Change CFLAGS once, every build follows.

Rules: target, prerequisites, recipe

“basics depends on 01_basics.c; if the source is newer than the executable, run this command.” all is the first rule, so plain make means make all. # starts a comment.

Flag	Meaning
-std=c11	The 2011 version of the language
-Wall -Wextra	Most warnings, then some more. Always.
-O3	Optimize hard. Can change what undefined behavior does — PRINT17
-march=native	Use this CPU’s instructions; the executable may not run elsewhere

What make does

Terminal — four runs in a row

```
$ make
gcc ./01_basics.c -std=c11 -Wall -Wextra -O3 -march=native -o basics
./01_basics.c:184:14: warning: unused variable 'flower' [-Wunused-variable] (and nine more)
gcc ./02_array_pointers.c -std=c11 -Wall -Wextra -O3 -march=native -o array_pointers
./02_array_pointers.c:16:31: warning: initialization of 'int *' from incompatible pointer type 'int (*)[3]'
$ make
make: 'all' is up to date.
$ touch 02_array_pointers.c
$ make
gcc ./02_array_pointers.c -std=c11 -Wall -Wextra -O3 -march=native -o array_pointers
$ make basics
make: 'basics' is up to date.
```

Timestamps decide

make compares the date of each target with its prerequisites. touch makes a file “newer”, so only that rule runs again.

One target at a time

make basics builds that rule only. Plain make builds the first target, all, and everything it depends on.

Why bother

With two files it saves typing. With two hundred, it saves hours: only the .c files you edited are recompiled.

TRY IT · CHECKPOINT 3 · Makefile

Try it — Makefile

Change — line 12: replace the TAB before $(CC) by eight spaces. Run make.

```
Makefile:12: *** missing separator (did you mean TAB instead of 8 spaces?). Stop.
```

Why — a recipe must start with a real TAB. The oldest quirk of make, and the first thing to check when it refuses to run.

Change — add **SRC3 = ./03_char_strings.c**, **EXE3 = char_strings**, put $(EXE3) in the all line, and copy a third rule.

```
make builds three executables; a second make says 'all' is up to date
```

Why — every program of the course can get a rule. The pattern is always the same three lines.

Change — append a rule **clean:** whose recipe is **rm -f $(EXE1) $(EXE2)**. Run make clean, then make.

```
rm -f basics array_pointers — then both gcc lines run again
```

Why — a target with no file of that name is just a named command. Declare it with **.PHONY: all clean** so a file called clean can never confuse make.

Change — in CFLAGS, -O3 becomes -O0. Run make.

```
make: 'all' is up to date.
```

Why — the Makefile itself is not a prerequisite, so nothing looks out of date. make clean first (or touch the .c files), make again, and ./basics shows the garbage PRINT17 of checkpoint 2.

PART 3 OF 6

02_array_pointers.c

arr, &arr[0] and &arr: one address, three types — and why a function can change your array.

array decay | int(*)[3] | pointer arithmetic | passing arrays | memcpy

```
$ make array_pointers && ./array_pointers
```

Two warnings are expected. Both are part of the lesson.

02_array_pointers.c · LINES 6–13

An array name is an address

02_array_pointers.c

```c
int arr[3] = {11, 12, 13};

printf("PRINT arr: %d, %p\n", arr, arr);
/* arr is actually a pointer on the first value */


int* first_val_ptr = &arr[0];
printf("FIRST VALUE POINTER (&arr[0]): %p\n",
    first_val_ptr);
printf("SAME BUT DEREFERENCED: %d\n",
    *first_val_ptr);
```

Terminal — compile, then run

```
warning: format '%d' expects argument of
type 'int', but argument 2 has type 'int *'
PRINT arr: 699821432, 0x7ffc29b66d78
FIRST VALUE POINTER (&arr[0]): 0x7ffc29b66d78
SAME BUT DEREFERENCED: 11
```

arr “decays” to &arr[0]

In almost any expression, an array name is converted to a pointer to its first element. That is why both %p lines print the same address.

The warning is right

699821432 is a slice of the address read as an int: meaningless. An address is printed with %p, portably as (void*)arr. Checkpoint 4 fixes it.

02_array_pointers.c · LINES 15–25

&arr has a different type

02_array_pointers.c

```c
/* Q: What's wrong with the two lines below? */
int* entire_arr_ptr = &arr;
printf("TEST &arr: %p\n", entire_arr_ptr);
/* TWO DIFFERENT TYPES, SAME VALUE!! */
printf("\nSUMMARY: arr: %p, &arr[0]: %p,"
       " &arr: %p\n", arr, &arr[0], &arr);
```

Terminal — the SUMMARY is one line on screen

```
warning: initialization of 'int *' from
incompatible pointer type 'int (*)[3]'
TEST &arr: 0x7ffc29b66d78
SUMMARY: arr: 0x7ffc29b66d78,
         &arr[0]: 0x7ffc29b66d78,
         &arr: 0x7ffc29b66d78
```

Expression	Type	Meaning
arr	int[3], decays to int*	The array; as a value, its first element's address
&arr[0]	int*	Pointer to the first element
&arr	int (*)[3]	Pointer to the whole array of 3 ints

Same number, different arithmetic

arr + 1 moves 4 bytes, to the second int. &arr + 1 moves 12 bytes, past the whole array. The type tells the compiler how far one step is.

The warning is how you ask C for a type

There is no typeof() to print. Assign to a pointer of the wrong type, and the "incompatible pointer type" message names both types for you.

02_array_pointers.c · LINES 28–39

A pointer to the whole array

```c
int(* arr_ptr)[] = &arr; // it works also with [3]
printf("\nDEREFERENCED ARRAY POINTER WITH INDEX: %d\n", (*arr_ptr)[1]); // value at the index i
printf("DEREFERENCED ARRAY POINTER (WITHOUT INDEX): %p\n", *arr_ptr); // first value pointer (int*)
printf("DEREFERENCED x2 ARRAY: %d\n", **arr_ptr); // first value
printf("ENTIRE ARRAY POINTER?? %p\n", arr_ptr); // NO!! ;-) first value pointer (type: int(*)[3])
printf("NOTE: POINTER TO POINTER: %p\n", &arr_ptr);
```

```
DEREFERENCED ARRAY POINTER WITH INDEX: 12
DEREFERENCED ARRAY POINTER (WITHOUT INDEX): 0x7ffc29b66d78
DEREFERENCED x2 ARRAY: 11
ENTIRE ARRAY POINTER?? 0x7ffc29b66d78
NOTE: POINTER TO POINTER: 0x7ffc29b66d70
```

`(*arr_ptr)[1]`

Go to the array, take element 1: 12. Parentheses matter, as before.

`*arr_ptr and **arr_ptr`

The array (decays to int*, same address), then element 0: 11.

`&arr_ptr`

Where the pointer variable itself lives, 8 bytes below the array.

02_array_pointers.c · LINES 3, 42–57

Arrays are passed by address

02_array_pointers.c

```
void modifyArray(int any_array[3]);
printf("\nARRAY BEFORE: %d\n", arr[1]);
modifyArray(arr);
printf("ARRAY AFTER: %d\n", arr[1]);
/* To copy array (including string): */
//memcpy(array2, array1, sizeof(array2))

void modifyArray(int any_array[3]){
    any_array[1] = 999;
}
```

```
ARRAY BEFORE: 12
ARRAY AFTER: 999
```

int any_array[3] really means int* any_array

The [3] is documentation; the function receives the address of arr[0] and writes straight into the caller's array. No & was needed: arr decayed on its own.

Nothing is copied — unless you copy it

In Python the function would have worked on a copy of the list. Here a copy is explicit: memcpy(copy, arr, sizeof(arr)) from <string.h>, or a loop.

So a function cannot know the length

Inside modifyArray, sizeof(any_array) is 8: the size of a pointer. Real code passes the length as a second parameter: void f(int* a, size_t n).

TRY IT · CHECKPOINT 4 · 02_array_pointers.c

Try it — array pointers

Change — line 16: remove the &, so it reads **int* entire_arr_ptr = arr;**

```
no more “incompatible pointer type” warning; same address printed
```

Why — arr decays to int*, which matches the declaration. &arr is an int (*)[3] and does not. Line 8 too: print it as %p with (void*)arr and the other warning goes.

Change — line 37: uncomment **int(* tab)[] = *arr_ptr;**

```
warning: initialization of 'int (*)[]' from incompatible pointer type 'int *'
```

Why — *arr_ptr is the array, which decays to int*; tab wants a pointer to an array. The mismatch of the previous slide, mirrored — and once again the message names both types.

Change — line 11: **&arr[1]** instead of &arr[0].

```
FIRST VALUE POINTER (&arr[0]): 0x…6d7c — SAME BUT DEREFERENCED: 12
```

Why — the address ends in 7c instead of 78: four bytes further, one int. Pointer arithmetic counts in elements, never in bytes.

Change — in modifyArray, add **any_array[5] = 1;** Compile with the Makefile’s flags, run.

```
warning: array subscript 5 is outside array bounds of 'int[3]' — then *** stack smashing detected ***: terminated
```

Why — the function wrote past arr, into main’s stack frame. Without -O3 the warning may not appear and the program may seem fine: silent undefined behavior again.

PART 4 OF 6

03_char_strings.c

There is no string type. There are characters, arrays of them, pointers to them, and a library that knows where they end.

char, char*, char[] · the \0 terminator · sizeof vs strlen · strcpy · strtol · one crash on purpose

```
$ gcc 03_char_strings.c -Wall -Wextra -o char_strings && ./char_strings
```

Two warnings are expected here too; one of them is the subject of a whole slide.

03_char_strings.c · LINES 6–9

Three ways to hold characters

```
char myOneChar = 'A'; // Single character + single quotes
char* myCharPointer = "ABCDE"; // One or more characters + double quotes: a string literal
char myCharArray[] = "VWXYZ"; // An array of char, filled from a literal
```

myCharArray — 6 bytes of your own, writable



sizeof gives 6: the invisible terminator counts

myCharPointer — 8 bytes holding an address



the literal lives in read-only memory

A char is a small number

'A' is 65, one byte. Single quotes: one character. Double quotes: a sequence ending in \0.

\0 is how a string ends

Nothing stores the length. printf, strlen and strcpy all read until they meet the terminator.

Array or pointer: the lesson of this file

Both print with %s. Only the array is yours to modify; the literal is not.

03_char_strings.c · LINES 11–19

Printing and pointer arithmetic

```c
/* Different types, but the compiler automatically converts the array into a pointer */
printf("CHAR POINTER: %s\n", myCharPointer);
printf("CHAR POINTER DEREFERENCED: %c\n", *myCharPointer);
printf("CHAR ARRAY: %s\n", myCharArray);

/* Pointer arithmetic */
char* secondCharPtr = myCharPointer + 1;
printf("CHAR POINTER +1: %s\n", secondCharPtr);
printf("CHAR POINTER +1 DEREFERENCED: %c\n", *secondCharPtr);
```

```
CHAR POINTER: ABCDE
CHAR POINTER DEREFERENCED: A
CHAR ARRAY: VWXYZ
CHAR POINTER +1: BCDE
CHAR POINTER +1 DEREFERENCED: B
```

%s starts at an address and stops at \0

Array or pointer, %s receives an address. %c receives one char: *myCharPointer is the first letter.

+ 1 means “one element further”

One char is one byte, so secondCharPtr points at B; %s from there prints BCDE. On an int* the same + 1 moved 4 bytes.

03_char_strings.c · LINES 21–24

Modifying a string

03_char_strings.c

```
/* Modify a character */
myCharArray[2] = '-';
//myCharPointer[2] = '-'; // Invalid:
// "Segmentation fault (core dumped)"
printf("MODIFIED CHAR ARRAY: %s\n",
myCharArray);
```

```
MODIFIED CHAR ARRAY: VW-YZ
```

The array is yours

myCharArray is six bytes on the stack, copied from the literal when the program started. Index 2 is X; it becomes a dash.

TRY IT NOW · STEP 1

Uncomment line 23. It compiles without a word. Run it.

```
Segmentation fault (core dumped) # echo $? gives
139
```

The literal "ABCDE" sits in read-only memory. Writing there is forbidden by the operating system, which kills the program.

TRY IT NOW · STEP 2

Line 8: declare it **const char* myCharPointer**. Recompile.

```
error: assignment of read-only location
'*(myCharPointer + 2)'
```

Same mistake, caught at compile time instead of run time. Declare pointers to literals const: the compiler refuses what the OS would have killed.

03_char_strings.c · LINES 26–32, 74, 78

Measuring a string

```c
int strlen1 = sizeof(myCharArray)/sizeof(myCharArray[0]);
printf("STRING LENGTH (CHAR ARRAY): %d\n", strlen1); // the invisible \0 is counted
int strlen2 = sizeof(myCharPointer)/sizeof(myCharPointer[0]); // not the good method for char*: 8 / 1 = 8
printf("STRING LENGTH (CHAR POINTER): %d\n", strlen2);
printf("STRING LENGTH (with strlen()): %ld\n", strlen(some_other_string)); // strlen() does not count the \0
```

Expression	Gives	Because
sizeof(myCharArray) / sizeof(myCharArray[0])	6	Bytes of the array, \0 included: 5 letters + 1
sizeof(myCharPointer) / sizeof(myCharPointer[0])	8	Bytes of a pointer (64-bit), whatever it points to
strlen(myCharArray)	5	Characters before the first \0 — the usual “length”
strlen(some_other_string), from {'x', 'x'}	2	Two x, then ten \0: it stops at the first one
strlen(...) after strcpy(..., "Hello world")	11	The space counts, the terminator does not

gcc flags the pointer case: warning: division 'sizeof (char *) / sizeof (char)' does not compute the number of array elements. And strlen returns a size_t, printed portably with %zu rather than %ld.

03_char_strings.c · LINES 36–45

Copying a string — the wrong way

03_char_strings.c

```
/* COPYING A STRING: THE WRONG WAY! */
char* some_string = "A";
some_string = myCharArray; // an address!
printf("\nCOPIED STRING: %s\n", some_string);
some_string[2] = '@';
printf("MODIFIED COPIED STRING: %s\n",
    some_string);
printf("CHAR ARRAY (SOURCE): %s\n",
    myCharArray); // modified as well!
```

```
COPIED STRING: VW-YZ
MODIFIED COPIED STRING: VW@YZ
CHAR ARRAY (SOURCE): VW@YZ
```

After the assignment, two names for the same six bytes



= copies the address, never the characters

some_string now points into myCharArray. Change one, you have changed the other. Python has the same trap with b = a on a list.

The right way: copy the bytes

strcpy(destination, source) from <string.h>, into an array of the destination's own that is large enough — next slide. memcpy does the same for any bytes.

03_char_strings.c · LINES 49–57, 69–78

Initialization and strcpy

03_char_strings.c

```c
/* These 3 are equivalent: */
//char some_other_string[11] = {0};
//char some_other_string[11] = {};
//char some_other_string[11] = "";

/* two 'x', followed by ten \0: */
char some_other_string[12] = {'x', 'x'};

printf("\nSTRING VALUE: %s\n",
    some_other_string);
strcpy(some_other_string, "Hello world");
printf("STRING VALUE AFTER strcpy: %s\n",
    some_other_string);
```

```
STRING VALUE: xx
STRING LENGTH: 12
STRING LENGTH (with strlen()): 2
STRING VALUE AFTER strcpy: Hello world
STRING LENGTH (with strlen()): 11
```

TRY IT NOW

Line 75: copy **"Hello world!"** instead — 12 letters plus \0, into 12 bytes.

```
warning: … forming offset 12 is out of the bounds
[0, 12]
*** buffer overflow detected ***: terminated
```

strcpy copies until it meets \0, into whatever lies there. The destination must be an array you own, and large enough: count the terminator. The abort comes from glibc's runtime check; without it, silent corruption.

03_char_strings.c · LINES 91–113

Conversions and the rest of string.h

03_char_strings.c

```c
/* Convert string to int (and vice versa) */
char str_number[5] = "1000"; // count the \0!
char* char_ptr;
long converted_int =
    strtol(str_number, &char_ptr, 10); // base 10

printf("\nCONVERTED TO INT: %ld\n",
    converted_int);
/* atoi() exists, but avoid it: no error check */

/* strtof() and strtod() for float and double */

/* the other way round: */
//sprintf(str, "%d", 42);
```

```
CONVERTED TO INT: 1000
```

Function	Does
strlen(s)	Characters before \0
strcpy(dest, src)	Copies src into dest, with \0
strncpy(dest, src, n)	At most n bytes, maybe no \0
strcat(dest, src)	Appends src to dest
strcmp(a, b)	0 when equal; never use ==
memcpy(dest, src, n)	Copies n raw bytes
memset(s, c, n)	Fills n bytes with c
strtol / strtod	Text to number, reports the stop
sprintf / snprintf	Number to text, into an array

The first seven come from <string.h>, strtol and strtod from <stdlib.h>, sprintf from <stdio.h>. The manual is one command away: man 3 strtol.

TRY IT · CHECKPOINT 5 · 03_char_strings.c

Try it — strings

Change — line 17: **myCharPointer + 5** instead of + 1.

```
CHAR POINTER +1: (nothing) — DEREFERENCED: an invisible character
```

Why — + 5 lands exactly on the terminator: %s prints an empty string and %c prints \0. With + 6 you are past the literal: undefined behavior.

Change — line 75: **strncpy(some_other_string, "Hello world!", 12);**

```
warning: output truncated before terminating nul — yet it prints Hello world! and strlen says 12
```

Why — twelve bytes copied, no \0 written. It looks right only because the byte after the array happened to be zero. Luck is not a terminator.

Change — line 92: **char str_number[6] = "12abc";** and print char_ptr with %s after the conversion.

```
CONVERTED TO INT: 12 — rest: abc
```

Why — strtol stops at the first non-digit and stores that position in char_ptr. atoi would have returned 12 and told you nothing: that is why the file says to avoid it.

Change — add **char buf[16]; sprintf(buf, "%d", 42); printf("%s\n", buf);**

```
42
```

Why — sprintf is printf into an array: the reverse of strtol. Prefer snprintf(buf, sizeof buf, "%d", 42), which cannot overflow buf.

PART 5 OF 6

Arguments and files

Three short programs that talk to the outside world: 04_argv.c, 05_write_files.c and 06_file_lines_to_string_array.c.

argc, argv | FILE* | fopen, fprintf, fclose | fgets | exit status

```
$ gcc 04_argv.c -Wall -Wextra -o argv && ./argv hello world 42
```

06 expects a file named some_text.txt next to the executable: create it before running.

04_argv.c · THE WHOLE FILE

Command-line arguments

04_argv.c

```c
#include <stdio.h>

int main (int argc, char *argv[]) {
    int i=0;
    printf("\ncmdline args count=%d", argc);
    /* First argument is executable name only */
    printf("\nexe name=%s", argv[0]);

    for (i=0; i< argc; i++) {
        printf("\narg%d=%s", i, argv[i]);
    }
    printf("\n");
    return 0;
}
```

```
$ ./argv hello world 42
cmdline args count=4
exe name=./argv
arg0=./argv
arg1=hello
arg2=world
arg3=42
```

argv is an array of strings, argc its length

The shell splits the line at spaces and passes the pieces. argv[0] is the program, so three words give 4.

Everything arrives as text

"42" is a char*, not a number. strtol(argv[3], &end, 10) turns it into one.

TRY IT · CHECKPOINT 6 · 04_argv.c

Try it — arguments

Change — nothing in the code. Run **./argv** with no argument at all.

```
cmdline args count=1 — arg0=./argv and nothing else
```

Why — the program name is always there. argv[argc] is NULL: never read argv[1] before checking that argc is at least 2.

Change — run **./argv "hello world"**, then **./argv *.c**

```
count=2, arg1=hello world — then count=8, one .c file per argument
```

Why — the shell splits, quotes and expands wildcards before main even starts. C receives the result and never sees the quotes or the star.

Change — after the loop, add **long n = strtol(argv[1], NULL, 10); printf("\ndouble: %ld", 2*n);** with #include <stdlib.h>. Run **./argv 21**.

```
double: 42
```

Why — arguments are text; strtol makes a number of argv[1]. The NULL means “I do not need to know where parsing stopped”.

Change — same program, run with no argument.

```
Segmentation fault (core dumped)
```

Why — argv[1] is NULL and strtol dereferences it. Guard it: if (argc < 2) { fprintf(stderr, "usage: argv N\n"); return 1; } — the exit status 1 tells the shell something went wrong.

05_write_files.c · THE WHOLE FILE, ONE FIX

Writing a file

05_write_files.c

```c
#include <stdio.h>
int main(){ // original: void main()
    FILE *f = fopen("file.txt", "w");
    if (f == NULL){
        printf("Error opening file!\n");
        return 1;
    }
    const char *text = "Write this to the file";
    fprintf(f, "Some text: %s\n", text);
    int i = 1; float pi = 3.1415927;
    fprintf(f, "Integer: %d, float: %f\n", i, pi);
    char c = 'A';
    fprintf(f, "A character: %c\n", c);
    fclose(f);
    return 0;
}
```

```
$ gcc 05_write_files.c -Wall -Wextra -o write_files

$ ./write_files
$ cat file.txt
Some text: Write this to the file
Integer: 1, float: 3.141593
A character: A
```

fprintf is printf with a destination

Same format strings, first argument a FILE*. Nothing appears on screen; cat shows the result. fclose flushes the buffer and releases the file.

Why the one fix

The original has void main() and exit(1). gcc answers: warning: return type of 'main' is not 'int' [-Wmain]. main returns the exit status; int main with return 1 says the same thing properly.

06_file_lines_to_string_array.c · LINES 5–28

Reading a file line by line

MAX_LINES is 100, MAX_LEN is 1000

```c
int main(void){
    char data[MAX_LINES][MAX_LEN];
    FILE *file = fopen("some_text.txt", "r");
    if (file == NULL){
        printf("Error opening file.\n");
        return 1;
    }
    int line = 0;
    while (!feof(file) && !ferror(file))
        if (fgets(data[line], MAX_LEN, file) != NULL)
            line++;
    fclose(file);
    for (int i = 0; i < line; i++)
        printf("%s", data[i]);
    return 0;
}
```

```
$ printf 'First line\nSecond\nThird\n' >
some_text.txt
$ ./file_lines
First line
Second
Third
$ rm some_text.txt; ./file_lines; echo $?
Error opening file.
1
```

A 2D char array: 100 lines of 1000 characters

data[line] is one row, a char array of MAX_LEN. fgets fills it with at most MAX_LEN − 1 characters, keeps the \n, adds \0, and returns NULL when nothing is left — which, with feof, ends the loop. printf("%s") then prints each row, newline included.

TRY IT · CHECKPOINT 7 · 05_write_files.c AND 06_file_lines_to_string_array.c

Try it — files

Change — run ./write_files twice, cat file.txt. Then replace **"w"** by **"a"** in fopen, rebuild, run twice again.

```
3 lines after two runs with "w" — 6 lines after two runs with "a"
```

Why — "w" truncates the file every time it opens it; "a" appends. "r" reads and fails if the file is missing: that is what 06 relies on.

Change — put **void main()** back, as in the original file, and compile with -Wall.

```
warning: return type of 'main' is not 'int' [-Wmain]
```

Why — the standard defines main as returning an int, the exit status the shell reads with echo $?. gcc still builds it; other compilers refuse.

Change — in 06: **#define MAX_LEN 10**, and print each row as **printf("%d: %s\n", i, data[i]);**

```
0: First lin — 1: e of the — 2: file … every line chopped into 9-character pieces
```

Why — fgets reads at most MAX_LEN – 1 characters and keeps one byte for \0. The next call simply continues the same line; nothing is lost, only split.

Change — give 06 too many lines: **seq 1 150 > some_text.txt**, run it.

```
it never ends (Ctrl-C) — with -fsanitize=address: stack-buffer-overflow … in fgets
```

Why — the 101st fgets writes past data[99] and corrupts the variables next to it, the loop counter included. Fix the condition: **&& line < MAX_LINES**.

PART 6 OF 6

07_example_malloc_realloc.c

Memory whose size is decided while the program runs — and the one warning, one crash and one silent bug that come with it.

stack vs heap | malloc | realloc | free | AddressSanitizer

```
$ gcc 07_example_malloc_realloc.c -Wall -Wextra -O3 -o malloc_realloc && ./malloc_realloc
```

Keep -O3 for this one: two of the warnings only appear when the optimizer looks closely.

07_example_malloc_realloc.c · TWO KINDS OF MEMORY

Static vs dynamic memory

THE STACK — STATIC ALLOCATION

```
int array2[] = {11, 12, 13};
```

Size fixed at compile time. Freed automatically when the function returns. Fast, small (a few MB), cannot grow.

Every array of Parts 1 to 5 lived here.

THE HEAP — DYNAMIC ALLOCATION

```
int* array = malloc(3 * sizeof(int));
```

Size decided while the program runs. Lives until you call free. As big as the machine allows; can grow with realloc.

You get back an address; the memory is contiguous and uninitialized.

From <stdlib.h>	What it does
void* malloc(size_t bytes)	Reserves bytes on the heap, returns their address or NULL. No cast needed
void* calloc(count, size)	Same, for count × size bytes, all set to zero
void* realloc(p, bytes)	Resizes a block, moving it if needed: content kept, address may change
void free(p)	Returns the block. Forget it: a leak. Call it twice: a crash

07_example_malloc_realloc.c · LINES 5–20

malloc and the missing bounds check

07_example_malloc_realloc.c

```c
/* malloc: dynamic allocation (contiguous) */
int* array = malloc(3 * sizeof(int));
array[2] = 5;
printf("FIRST PRINT: %d\n", array[2]);

/* About the bound checking... */
array[30] = 10;
printf("BEFORE1: %d\n", array[30]);

/* a static array, for comparison */
int array2[] = {11, 12, 13};
array2[30] = 10;
printf("ALTBEFORE2: %d\n", array2[29]);
printf("BEFORE2: %d\n", array2[30]);
```

```
warning: array subscript 30 is outside array
bounds of 'int[3]' [-Warray-bounds=]
# … and the same for array2, subscript 29
FIRST PRINT: 5
BEFORE1: 10
ALTBEFORE2: 884924918
BEFORE2: 10
```

Heap or stack, nothing checks the index

120 bytes past a 12-byte block, and it reads back as 10. That it “works” is the trap.

ALTBEFORE2 is whatever was there

Never written: a stale stack value, different on your machine. gcc saw both bugs but can only warn.

07_example_malloc_realloc.c · LINES 22–27

Growing with realloc

07_example_malloc_realloc.c

```
array = realloc(array,
    4000000 * sizeof(int)); // 16 MB
//array2 = realloc(array2,
//    4000000 * sizeof(int));
array[3000000] = 10;
printf("AFTER1: %d\n", array[3000000]);
printf("AFTER2: %d\n", array[3150002]);
```

```
AFTER1: 10
AFTER2: 0
```

Missing from the file: free(array); at the end. The OS reclaims everything at exit, but in a long-running program every malloc needs its free.

TRY IT NOW

Uncomment line 23 and try to realloc the static array2.

```
error: assignment to expression with array type
```

array2 is not a pointer that can be pointed elsewhere: it is the three ints themselves, on the stack, with a size fixed forever. Only heap memory can grow.

From 12 bytes to 16 megabytes

realloc keeps the first three ints and may move the block, so always assign its result back to the pointer. AFTER2 is 0 by luck: fresh pages arrive zeroed, but realloc promises nothing about the new part. Want zeros? calloc, or memset.

TRY IT · CHECKPOINT 8 · 07_example_malloc_realloc.c

Try it — dynamic memory

Change — uncomment lines 13–14: **array[3000000] = 10;** before the realloc.

```
Segmentation fault (core dumped)
```

Why — 12 MB past a 12-byte block is memory the process does not own, so the OS kills it. Index 30 got lucky; index 3,000,000 did not. Same bug, different symptom.

Change — the flags: **gcc 07_example_malloc_realloc.c -g -fsanitize=address -o asan && ./asan**

```
ERROR: AddressSanitizer: heap-buffer-overflow … WRITE of size 4 … in main 07_example_malloc_realloc.c:11
```

Why — AddressSanitizer checks every memory access while the program runs and names the exact line. Try it on each program of this course that “worked”.

Change — line 6: **calloc(3, sizeof(int))** instead of malloc, and print array[1] before writing anything.

```
0 — guaranteed, where malloc would give anything
```

Why — calloc zeroes the block; malloc hands it over as is, like the uninitialized boss1 of Part 1.

Change — add **free(array);** before return 0. Then add it a second time.

```
warning: pointer 'array' used after 'free' [-Wuse-after-free] — then a crash at run time
```

Why — the block was already returned; the second free corrupts the allocator. One free per malloc, then set the pointer to NULL if it lives on.

WRAP-UP

What the messages mean

You see	It means	Seen in this course
warning: ...	Builds anyway. Read it, then fix it	Nearly every file
error: ...	Not valid C; no executable produced	const char*, realloc(array2)
Segmentation fault — exit 139	A pointer went where the process may not	writing a literal, array[3000000]
buffer overflow / stack smashing — 134	glibc caught an overflow at run time	strcpy of 13 bytes into 12
Nothing — wrong or lucky output	Undefined behavior: anything may happen	PRINT17, my_arr1[30], AFTER2
make: *** missing separator	A recipe line that does not start with TAB	Makefile, checkpoint 3
make: 'all' is up to date	Nothing is newer than its target	Makefile, after editing CFLAGS

`-Wall -Wextra -O2`
At compile time. Some checks only run when the optimizer is on.

`-g -fsanitize=address`
At run time. Names the line of every bad memory access.

`valgrind ./prog`
Leaks and invalid reads, no recompilation needed.

`gdb ./prog`
run, then bt after a crash: the stack of calls that led there.

WRAP-UP

What each file taught

01_basics.md

Compile, link, run. Types convert silently; cast when you mean it.

01_basics.c

Prototypes, & and *, by value vs by reference, structs and ->. Initialize everything.

Makefile

Targets, prerequisites, TAB recipes. Timestamps decide what is rebuilt.

02_array_pointers.c

An array name decays to its first address. Functions receive that address, never a copy.

03_char_strings.c

A string is chars up to \0. Literals are read-only; copy with strcpy, never with =.

04_argv.c

argc counts, argv holds text. Check argc, convert with strtol.

05_write_files.c

fopen, check NULL, fprintf, fclose. "w" truncates, "a" appends.

06_file_lines_to_string_array.c

fgets into a 2D char array. Fixed sizes need guards: line < MAX_LINES.

07_example_malloc_realloc.c

malloc, realloc, free. No bounds checks anywhere: let AddressSanitizer look.

WRAP-UP

Going further

PUT THE FILES TOGETHER

1. Lengths of a file

In 06, print each line with its strlen. Mind the \n that fgets keeps.

2. A file name from the command line

Replace "some_text.txt" by argv[1], with a usage message and exit status 1 when it is missing.

3. Lines without a limit

Replace the 100 × 1000 array by malloc'd rows, realloc the row table as the file grows, free everything at the end. Check it with AddressSanitizer.

LINKS FROM THE SOURCE FILES

Undefined behavior — the idea behind half of the checkpoints

String literals and C string handling — the rest of string.h

Header files — what #include really does

A hash table in C — dictionaries, built from structs

malloc vs calloc — with examples

TP : l'en-tête string.h — reimplement the library yourself (in French)

Replacing a substring — there is no function for it; write one

```
$ man 3 fgets   # the manual, for every function met here
```